DAT32-1

Intro to Machine Learning

Albert School — 16 hours

Contents

Preface

You arrive here already able to handle data. In **Pandas I** (DAT12-3) and **Scraping, Pandas II & Algorithmic Thinking** (DAT22-1) you learned to load, clean, filter, group and reshape tabular data, and to reason about the cost of an operation. In **SQL & Data Querying** (DAT22-5) you learned to turn a business question into a query and read its answer. In **Software Engineering & Code Design** (DAT22-3) you learned to make code reproducible and testable with Git, virtual environments, classes and `pytest`. From mathematics you bring the mean and variance, the idea of a probability distribution, the derivative of a function, and matrix multiplication. This book assumes all of that and builds on it; it does not re-teach it.

What you do **not** yet have is a model — a thing that takes data it has never seen and predicts a number. That is what machine learning adds, and this course teaches it in an unusual way: through **one** model, linear regression, followed all the way from a raw table to a tuned, validated, reproducible pipeline. Every core idea in machine learning — loss functions, optimisation, overfitting, baselines, evaluation metrics, regularization, cross-validation, hyperparameters — is introduced at the moment it first becomes necessary in that single workflow, not as an entry in a glossary.

This is deliberate. Spend sixteen hours on one model understood completely and you do not learn "linear regression"; you learn the **shape** of every machine-learning project. When you later meet logistic regression, decision trees or neural networks, you will not be learning a new paradigm — you will be substituting a new model into a cycle you already own.

A word on method: every idea arrives as a concrete example first and the general statement second. We keep one running problem throughout — **predicting the monthly rent of an apartment from its characteristics** — so that each new tool refers back to numbers you already half-know. We start with a single feature, the surface area in square metres, and grow from there.

1 What machine learning is: taxonomy and workflow

Suppose you manage a rental portfolio and want to predict the monthly rent of a flat before it goes on the market. You could try to write the rule by hand — "start at €400, add €12 per square metre, add €150 if it has a balcony…" — but you would be guessing the numbers.

Machine learning instead **learns** those numbers from past rentals: you show an algorithm many (apartment, rent) pairs and it fits a model that reproduces them as closely as possible, then applies that model to new apartments.

Definition 1 **Machine learning** builds a model that maps inputs to an output by fitting it to data, rather than by being programmed with explicit hand-written rules. The fitted model is then used to predict the output for new, unseen inputs.

Before zooming in on our one model, here is the map of the territory — just enough to know where regression sits.

Definition 2 **Supervised learning** fits a model on data where every example carries a known target value. **Unsupervised learning** works on data with no target and instead discovers structure (groups, directions) in the inputs alone.

Definition 3 Within supervised learning, **regression** predicts a continuous numeric target (a rent, a temperature, a sales figure) and **classification** predicts a discrete category (spam / not-spam, which of five products). **Clustering** is the archetypal unsupervised task: it partitions examples into groups without any target.

The task type is decided by the target, not by the field or the difficulty. Ask: **is there a target to predict, and is it a number or a category?** A number to predict is regression; a category is classification; no target, "find the natural groups," is clustering.

> **Worked example — naming the task.** "Predict tomorrow's electricity demand in megawatts" — a number, so **regression**. "Will this customer churn next month, yes or no?" — a category, so **classification**. "Group our 50 000 customers into segments for marketing, we don't know the segments in advance" — no target, so **clustering** (unsupervised). Rent prediction is regression: the target, euros per month, is continuous.

Whatever the model, a supervised-learning project follows the same cycle. The rest of this book **is** this cycle, walked once on linear regression.

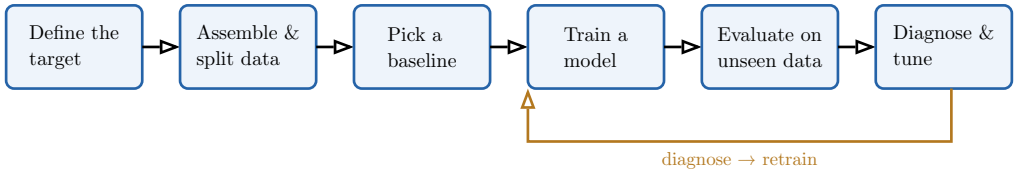


Figure 1: The supervised machine-learning workflow. It is model-independent: this course executes every step on linear regression, but the same loop drives every model you will meet later. The orange arrow is the heart of the craft — you rarely train once; you train, diagnose, and retrain.

2 Core vocabulary, built by hand

Rather than list definitions, we build the vocabulary by constructing the dataset our model will use.

> **Worked example — from a table to features and a label.** Here are five past rentals:

Surface (m²)	Rooms	Balcony	Rent (€/month)
30	1	no	620
45	2	no	780
60	2	yes	1010
75	3	yes	1180
90	3	yes	1300

Each **row** is an example. The columns **surface**, **rooms** and **balcony** are the inputs we predict from; the column **rent** is what we want to predict.

Definition 4 A **feature** is an input variable used to predict the target (surface, rooms, balcony). A **label** is the known target value for a given example (the rent). A supervised dataset is a table of examples, each row pairing a feature vector with its label.

We write one example as a feature vector $x = (x_1, ..., x_p)$ with label y, and the whole dataset as n pairs $(x^{(i)}, y^{(i)})$ for $i = 1, ..., n$. In the table above, $n = 5$ and $p = 3$.

Now the subtle part. If we judge the model on the very apartments it was fitted to, we measure memorisation, not prediction. So we **split** the data.

Definition 5 The **training set** is the data the model is fitted on. The **validation set** is held out from training and used to compare models and tune choices. The **test set** (also called the **holdout set**) is used exactly once, at the very end, to estimate performance on unseen data. An example used to choose or tune the model must never also be used to report its final performance.

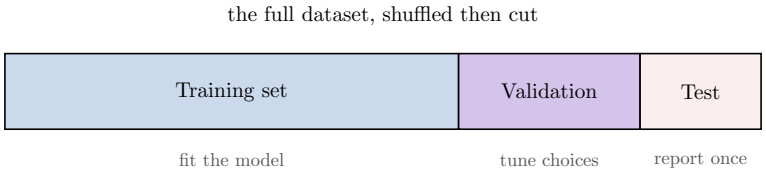


Figure 2: The three-way split. A typical division is 60/20/20, but the exact proportions matter less than the discipline: the test set is sealed away until the very end. Touch it earlier and it can no longer tell you the truth.

Common pitfall The single most common way to fool yourself in machine learning is to let the test set influence the model — even indirectly, by peeking at its score while choosing a setting. A model evaluated on data it has effectively already seen looks far better than it is. Split first, then forget the test set exists until the final line of the project.

3 Simple linear regression from first principles

Start with one feature. Plot rent against surface for our five apartments and the points fall almost on a straight line — bigger flats cost more, at a roughly constant rate per square metre. A model that captures exactly this is the simplest useful one.

Definition 6 **Simple linear regression** models the target as a linear function of one feature:

$$\hat{y} = \beta_0 + \beta_1 x,$$

where β_0 is the **intercept** and β_1 is the **slope**. The hat on $\hat{y}$ marks it as the model's prediction, distinct from the true label y.

For any choice of (β_0, β_1) the line misses each point by some vertical gap, the **residual** $y^{(i)} - \hat{y}^{(i)}$. Fitting the model means choosing the line that makes these gaps collectively smallest — but "smallest" needs a precise meaning, and that is the job of a cost function.

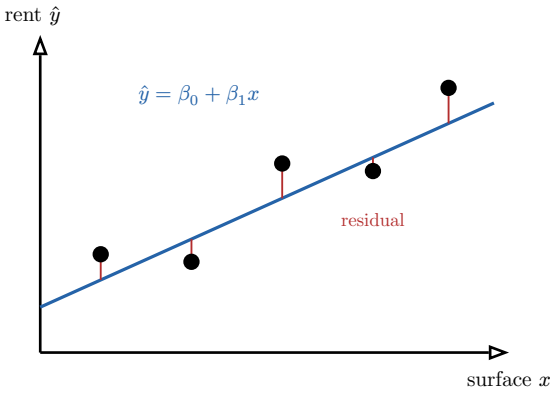


Figure 3: Simple linear regression on the rent data. The blue line is the model; each red segment is a residual, the vertical gap between a real rent (black dot) and the line's prediction. Ordinary least squares chooses the line that minimises the sum of the **squared** residual lengths.

Definition 7 The **residual sum of squares** (RSS) measures total miss as the sum of squared residuals over the training set:

$$\mathrm{RSS}(\beta_0, \beta_1) = \sum_{i=1}^{n} \left(y^{(i)} - \beta_0 - \beta_1 x^{(i)}\right)^2.$$

Why **squared** and not, say, absolute? Squaring makes the cost a smooth bowl with a single lowest point, which can be minimised exactly by calculus, and it penalises large misses disproportionately. Setting the two partial derivatives of the RSS to zero and solving gives a closed-form answer.

Definition 8 **Ordinary least squares** (OLS) chooses the coefficients that minimise the RSS. For simple regression the minimiser is available analytically:

$$\beta_1 = \frac{\sum_{i=1}^{n} \left(x^{(i)} - \bar{x}\right)\left(y^{(i)} - \bar{y}\right)}{\sum_{i=1}^{n} \left(x^{(i)} - \bar{x}\right)^2}, \quad \beta_0 = \bar{y} - \beta_1 \bar{x},$$

where $\bar{x}$ and $\bar{y}$ are the means of the feature and the label.

Worked example — the slope is a familiar quantity. The numerator of β_1 is the sample covariance of x and y (times n), and the denominator is the sample variance of x (times n). So $\beta_1 = \mathrm{Cov}(x, y) / \mathrm{Var}(x)$ — the slope is covariance normalised by the spread of the feature. The

mean, variance and covariance you met in statistics are exactly the ingredients of the least-squares line.

Interpreting the coefficients. The slope β_1 is the predicted change in rent for a one-square-metre increase in surface; if the fit gives $\beta_1 = 9.2$, each extra square metre adds about €9.20 to the predicted monthly rent. The intercept β_0 is the predicted rent at surface zero — often not literally meaningful (no flat has zero area), but necessary to anchor the line's height.

Common pitfall A regression coefficient is an **association in this dataset**, not a proven cause. "Each square metre adds €9.20" describes the fitted line; it does not prove that enlarging a specific flat will raise its rent by that amount, because surface is entangled with location, age and everything else that varies alongside it.

OLS gives a line for **any** data, but for its coefficients to support trustworthy **inference** — confidence intervals, significance — the data must satisfy modelling assumptions.

Definition 9 Valid statistical inference on OLS coefficients requires: the relationship is **linear** in the coefficients; the errors have **zero mean** and **constant variance** (homoscedasticity); the errors are **uncorrelated** with one another; and they are **independent of the features**. When these hold, the OLS estimates are unbiased.

4 Baselines and evaluation metrics

We have a fitted line. Is it any good? Two questions hide here — **is it better than doing nothing?** and **how wrong is it, in numbers I can act on?** — and each needs its own tool.

4.1 The baseline: the floor you must beat

Worked example — the dumbest defensible predictor. Ignore every feature and always predict the **average** training rent, €978. This model is surely crude — yet any real model must beat it, or it has learned nothing. A model that ties the "always predict the mean" predictor has extracted no signal from surface, rooms or balcony.

Definition 10 A **baseline** is the simplest defensible predictor, used as the floor a model must clear. For regression the standard baseline is the **mean predictor**, which ignores the features and predicts the mean training label for every example. A model that does not beat the baseline has no value.

4.2 Metrics: how wrong, in what units

Definition 11 For predictions $\hat{y}^{(i)}$ against labels $y^{(i)}$ over n examples:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|y^{(i)} - \hat{y}^{(i)}\right|, \quad \text{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)} - \hat{y}^{(i)}\right)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}.$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}\left(y^{(i)} - \hat{y}^{(i)}\right)^2}{\sum_{i=1}^{n}\left(y^{(i)} - \bar{y}\right)^2}.$$

Read these in plain words. **MAE** (mean absolute error) is the average miss in euros; robust and easy to explain. **MSE** (mean squared error) squares each miss, so one €300 error counts far more than three €100 errors; its units are euros-squared, hard to interpret. **RMSE** takes the square root to return to euros while keeping that heavy penalty on large errors. $\boldsymbol{R^2}$ rescales the MSE against the baseline's error: it is the fraction of the rent's variance the model explains, where $R^2 = 0$ exactly matches the mean predictor and $R^2 = 1$ is a perfect fit (a negative R^2 means you did **worse** than the baseline).

> **Worked example — letting the business choose the metric.** A landlord who loses money in proportion to each euro of misquoted rent cares about **MAE** — every euro counts the same. A pricing team that can absorb small errors but is hurt badly by the occasional wildly wrong quote should optimise **RMSE**, which punishes big misses hardest. The metric is a business decision, not a default.

Common pitfall Always report the metric **on held-out data**. MAE or R^2 measured on the training set flatters the model — it can look excellent there and fail on new apartments. The baseline comparison and every metric belong on the validation or test set.

5 Gradient descent

OLS handed us the optimum in one formula. That is a luxury of linear regression with squared error; most models have no closed form, and even linear regression on millions of rows may be cheaper to solve iteratively. The general-purpose optimiser that underlies almost all of modern machine learning is **gradient descent**, and it is worth implementing here, on a problem where we can check it against the exact answer.

The idea is physical. The loss $L(\beta)$ is a landscape over the coefficients; training means walking downhill to its lowest point. At any position the gradient ∇L points in the direction of steepest **increase**, so stepping in the opposite direction goes downhill fastest.

Definition 12 **Gradient descent** minimises a loss $L(\beta)$ by repeatedly stepping against the gradient:

$$\beta \leftarrow \beta - \alpha \nabla L(\beta),$$

where $\alpha > 0$ is the **learning rate**. The steps repeat until the loss stops decreasing meaningfully — **convergence**.

For the MSE loss of simple regression the two partial derivatives are

$$\frac{\partial L}{\partial \beta_0} = -\frac{2}{n}\sum_{i=1}^{n}\big(y^{(i)} - \hat{y}^{(i)}\big), \quad \frac{\partial L}{\partial \beta_1} = -\frac{2}{n}\sum_{i=1}^{n}\big(y^{(i)} - \hat{y}^{(i)}\big)x^{(i)},$$

which is all you need to code the loop: predict, compute these two numbers, nudge β_0 and β_1, repeat.

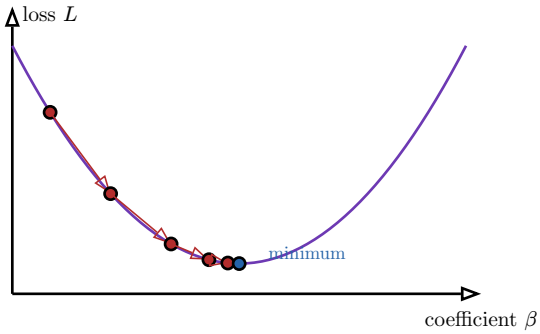


Figure 4: Gradient descent on a loss bowl. From a starting guess (left) each step moves against the gradient, downhill toward the minimum. Steps shrink as the slope flattens near the bottom — the algorithm naturally slows as it converges.

Definition 13 The **learning rate** α sets the step size and is the parameter that makes or breaks training. Too small, and convergence crawls, taking thousands of steps. Too large, and the steps overshoot the minimum, bounce up the far wall, and the loss **diverges** — growing each iteration instead of shrinking.

Common pitfall A loss that grows every iteration is the unmistakable signature of a learning rate that is too large — not a bug in your gradient. Before re-deriving the derivative, divide α by ten and rerun. Conversely, a loss that barely moves after many steps usually means α is too small.

6 Underfitting and overfitting

Linear regression assumes a straight-line relationship. What if the truth bends — rent rising steeply for small flats then levelling off? We can add curvature by feeding the model powers of the feature, $x, x^2, ..., x^d$ — **polynomial regression** of degree d. This single knob, the degree, lets us watch the central tension of all machine learning play out.

Worked example — three fits to the same noisy data. Fit degree 1 (a line) to gently curved, noisy data and it is too rigid to follow the bend — it misses in a systematic way. Fit degree 12 and the curve threads through **every** point, including the noise, snaking wildly between them. Fit degree 3 and it follows the real bend while ignoring the jitter. Only the middle fit will predict new apartments well.

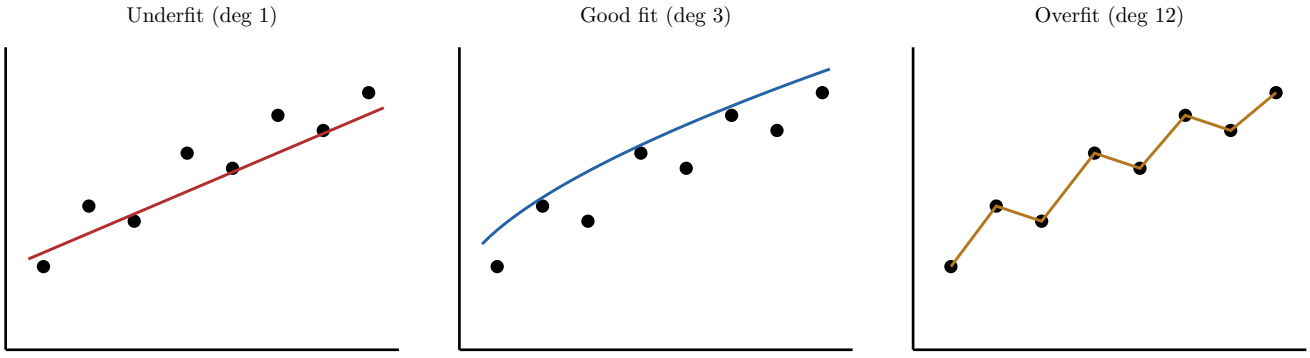


Figure 5: The same eight noisy points, three model complexities. Left: too rigid, missing the trend (high error everywhere). Right: so flexible it interpolates the noise (perfect on these points, useless on new ones). Centre: captures the signal, ignores the noise.

Definition 14 A model **underfits** when it is too simple to capture the structure in the data: it has high error on **both** the training set and unseen data. A model **overfits** when it is so flexible it fits the noise in the training set: **low** training error but **high** error on unseen data. The gap between training and unseen error is the fingerprint of overfitting.

You diagnose which regime you are in by plotting both errors as the model grows more complex (or as you add data).

Definition 15 A **learning curve** plots training error and validation error against model complexity (or training-set size). Underfitting appears as both curves high and close together; overfitting appears as a widening gap — training error low, validation error high. The sweet spot is where the validation error bottoms out.

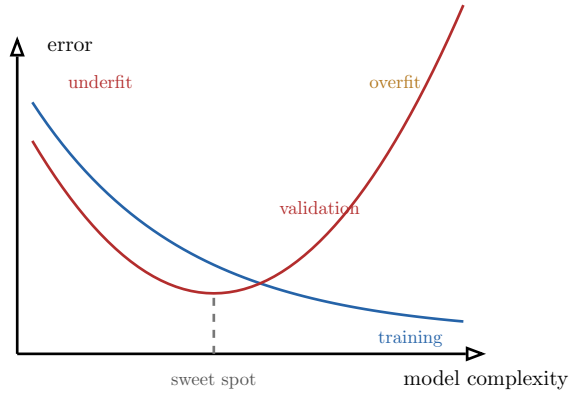


Figure 6: Learning curves against complexity. Training error (blue) always falls as the model grows more flexible. Validation error (red) falls, bottoms out, then rises as the model starts fitting noise. Choose the complexity at the bottom of the validation curve.

7 Regularization

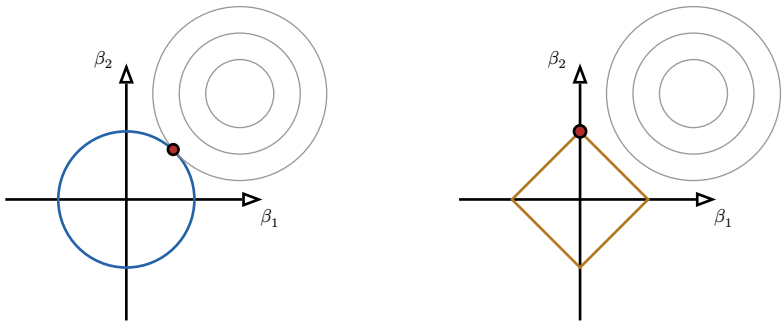
The overfitting cure need not be a coarser model. Overfit linear models betray themselves through **large, opposing coefficients** that let the curve swing wildly. **Regularization** tames this by adding the size of the coefficients to the loss, so the optimiser must trade fit against simplicity.

Definition 16 **Regularization** adds a penalty on coefficient size to the loss. **L2 regularization** (ridge) penalises the sum of squared coefficients; **L1 regularization** (lasso) penalises the sum of absolute coefficients:

$$L_{\text{ridge}} = \text{RSS} + \lambda \sum_{j=1}^{p} \beta_j^2, \quad L_{\text{lasso}} = \text{RSS} + \lambda \sum_{j=1}^{p} |\beta_j|.$$

The strength $\lambda \geq 0$ controls the trade-off: $\lambda = 0$ recovers plain OLS, and larger λ shrinks coefficients harder.

The two penalties differ in a consequential way: **L2 shrinks coefficients smoothly toward zero but rarely to exactly zero**, while **L1 drives some coefficients exactly to zero**, switching those features off entirely — it performs automatic feature selection. The reason is geometric: L1′s penalty region is a diamond with sharp corners on the axes, and the loss contours tend to first touch it at a corner, where a coordinate is zero.



L2 (ridge): round region
touch point off the axes — both β nonzero

L1 (lasso): diamond region
touch at a corner — $\beta_1 = 0$, feature dropped

Figure 7: Why L1 selects features and L2 does not. The grey ellipses are loss contours; the coloured shape is the region the penalty allows. The fit is the first contour to touch that region. L2′s circle is touched off the axes (both coefficients survive); L1′s diamond is usually touched at a corner, forcing a coefficient to exactly zero.

So: **prefer L1 (lasso)** when you suspect many features are irrelevant and want a sparse, interpretable model that names the few that matter; **prefer L2 (ridge)** when you believe most features contribute a little and you only want to keep their magnitudes under control — especially when features are correlated, where ridge is the steadier choice.

8 Cross-validation

The validation set we carved out has a weakness: its score depends on **which** apartments happened to land in it. An unlucky split — all the expensive flats in validation — gives a pessimistic, noisy estimate. With small datasets this luck of the draw can dominate.

> **Worked example — the same model, five different scores.** Split the data five different ways and you may get RMSE values of €95, €140, €110, €155 and €120 — for the **identical** model. Which is "the" performance? None alone; the honest answer is their average, and the spread warns you how uncertain that average is. That is precisely what cross-validation computes.

Definition 17 **k-fold cross-validation** splits the data into k equal folds. In turn, each fold is held out as the validation set while the model is trained on the other $k-1$ folds; this yields k scores, which are averaged. Every example is used for validation exactly once and for training $k-1$ times, so the estimate uses all the data and is far more stable than a single split.

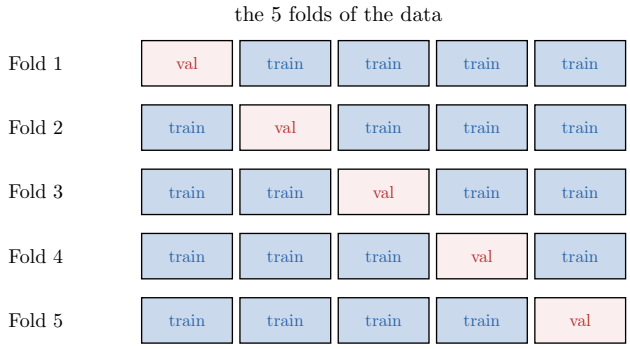


Figure 8: 5-fold cross-validation. Each row is one round: a different fifth of the data (red) is held out for validation while the rest (blue) trains the model. The five validation scores are averaged into one robust estimate, and every example is validated exactly once.

Cross-validation costs k times the training, but on the small-to-medium datasets typical of an intro course that cost is trivial and the gain in reliability is large. It is the default way to estimate performance and, as we see next, to tune.

9 Hyperparameters

We have quietly introduced several knobs that are **not** learned from the data by fitting: the polynomial degree d, the regularization strength λ, the learning rate α. These are different in kind from the coefficients.

Definition 18 A **parameter** is learned from the data during training (the coefficients β). A **hyperparameter** is a setting fixed **before** training that controls how training happens or how complex the model may be (λ, d, α). Parameters are **learned**; hyperparameters are **chosen**.

You choose a hyperparameter by trying candidate values, training a model for each, and comparing them — but comparing them **on the validation set**, or by cross-validation, never on the test set.

Worked example — tuning the regularization strength. To pick λ for ridge regression, fit the model for $\lambda \in \{0.01, 0.1, 1, 10, 100\}$, score each by 5-fold cross-validation, and keep the λ with the best average score. Only then, with λ fixed, do you unseal the test set for one final measurement.

Common pitfall Choosing a hyperparameter by its **test-set** score leaks the test set into the model: the test set has now influenced a modelling decision, so its score is no longer an honest estimate of performance on truly unseen data — it is optimistically biased. The validation set (or cross-validation) exists precisely so the test set can stay sealed. Tune on validation; report on test, once.

10 From simple to multiple regression

Surface alone leaves money on the table — rooms and balcony clearly matter too. Multiple regression uses them all at once.

Definition 19 **Multiple linear regression** predicts from several features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_p x_p.$$

Each coefficient β_j is the predicted change in the target for a one-unit increase in feature x_j **with all other features held fixed** — the "all else equal" clause is what makes multiple regression more than several simple ones stacked together.

That "all else equal" interpretation runs into trouble when features move together.

Worked example — when two features carry the same information. Add both **surface in m²** and **surface in ft²** as features. They are near-perfectly correlated — one is just the other times 10.76. The model cannot tell how much of the rent to credit to each, so it may land on huge offsetting coefficients (say +50 on one, −45 on the other) that fit equally well and flip sign if a single data point changes. The prediction is fine; the coefficients are nonsense.

Definition 20 **Multicollinearity** is strong correlation among features. It makes individual coefficients **unstable and uninterpretable** — small changes in the data swing them widely — because the data cannot separate the effect of one correlated feature from another. The model's predictions can stay accurate even as its coefficients become meaningless.

You diagnose multicollinearity by inspecting the correlation matrix of the features (looking for near-±1 pairs) and by watching for coefficients that are implausibly large, unstable across resamples, or carry the wrong sign. The remedy is usually to drop or combine the redundant features — or to lean on ridge regularization, which stabilises exactly this situation.

11 Reproducible workflows: scikit-learn pipelines

Everything so far — split, preprocess, fit, tune, evaluate — must be applied in the **same order** and **fitted on the training data only**, every time. Do it by hand across notebook cells and a subtle leak creeps in: you scale using statistics computed from the whole dataset, including validation and test, and your scores quietly inflate. The discipline you learned in **Software Engineering & Code Design** — reproducibility, clear contracts — has a direct instrument here.

Definition 21 A **scikit-learn pipeline** chains preprocessing steps and a model into one object. Calling `fit` runs each step's `fit` in order on the training data; calling `predict` applies each fitted transformation and then the model. Because every transformer is fitted only on the training portion, a pipeline **prevents data leakage** from validation or test data into preprocessing.

Worked example — the same object everywhere. A pipeline `[StandardScaler, Ridge]` is one object. Inside 5-fold cross-validation, scikit-learn re-fits the scaler on each fold's training part and applies it to that fold's validation part — automatically, correctly, with no leakage. In a manual workflow you would have to remember to do this nine separate times and would eventually forget once.

A pipeline is the **reproducible form of the entire course**: the identical chain of steps runs during cross-validation, during hyperparameter search, and at final test time, so the number you

report is the number a colleague re-running your code will get. That is where the ML project cycle and the engineering discipline meet — and where this course ends and every later model begins.

Exercises

1. For five stated business problems, name each as regression, classification, or clustering, and say whether it is supervised or unsupervised. Justify each by identifying the target (or its absence).

2. Take a labelled dataset, identify its features and label, and split it into training, validation and test sets. Explain in one sentence each what the three sets are for and why the test set must stay sealed.

3. Implement OLS simple linear regression from first principles using the analytical formulas (no library fitting). Interpret the slope and intercept in the units of the problem, and state the assumptions required for valid inference.

4. Implement gradient descent to fit the same regression. Plot the loss against iteration for three learning rates: one too small (slow), one well chosen, and one large enough to diverge. Relate each curve to the value of α.

5. Fit polynomials of degree 1 through 12 to a noisy dataset. Plot training and validation error against degree, identify the underfitting and overfitting regimes, and pick the degree at the bottom of the validation curve.

6. Implement the mean-predictor baseline. Compare a fitted model against it on held-out data and explain why a model that fails to beat the baseline has no value.

7. Compute MAE, MSE, RMSE and R^2 for a model on held-out data. Given two contrasting business contexts, select and justify a different metric for each.

8. Apply L1 (lasso) and L2 (ridge) regularization to a multi-feature regression. Report which coefficients each drives toward or exactly to zero, and give a situation where each penalty is the better choice.

9. Implement k-fold cross-validation by hand. Compare its averaged estimate, and the spread of its per-fold scores, against the estimate from a single train/test split.

10. Tune the regularization strength λ by cross-validation on the validation data. Then evaluate once on the test set. Explain precisely why tuning λ on the test set would invalidate the final number.

11. Extend the model to multiple features and interpret the coefficients "all else equal." Then add a near-duplicate feature, exhibit the resulting coefficient instability, and diagnose the multicollinearity from the correlation matrix.

12. Build a scikit-learn pipeline chaining a `StandardScaler` and a regularized regression, run it under cross-validation, and explain how the pipeline prevents preprocessing from leaking information out of the training folds.